DOCUMENTATION TECHNIQUE COMPLÈTE · V2.0

Bean Beacon — Cahier des charges

Application web de découverte de cafés. Interface React/Vite, API Express/TypeScript, base MongoDB, orchestration Docker Compose, pipeline GitLab CI/CD, supervision Prometheus · Grafana · Uptime Kuma.

React 18 + Vite + TypeScript | Node.js + Express | MongoDB + Mongoose | Docker Compose | GitLab CI/CD | Prometheus · Grafana | JWT Authentication | OpenStreetMap · Overpass

Bean Beacon Project

Architecture · API · DevOps · Déploiement

Table des matières

PARTIE I — VUE D'ENSEMBLE

§ 01 Présentation du projet 3

§ 02 Architecture système 3

§ 03 Schémas DevOps 4

PARTIE II — MISE EN ROUTE

§ 04 Prérequis 5

§ 05 Démarrage Docker Compose 5

§ 06 Tests et validations 6

PARTIE III — CI/CD ET DÉPLOIEMENT

§ 07 Workflow Git et GitLab 7

§ 08 Pipeline GitLab CI/CD 7

§ 09 Déploiement production 8

PARTIE IV — API REFERENCE

§ 10 Authentification 9

§ 11 Cafés 9

§ 12 Favoris 10

§ 13 Évaluations & Avis 10

§ 14 Tableau de bord API complet 11

PARTIE V — SUPERVISION & RÉFÉRENCE

§ 15 Supervision 12

§ 16 Livrables finaux 12

§ 17 Améliorations fonctionnelles 13

§ 18 Points d'attention & risques 14

§ 19 Problèmes fréquents 14

PARTIE I

Vue d'ensemble

Présentation, architecture système et schémas DevOps

§ 01

Présentation du projet

Bean Beacon est une application web de découverte de cafés. Elle combine un annuaire public avec recherche géolocalisée, profils et avis communautaires, soumission de cafés par les utilisateurs avec modération, recommandations basées sur les préférences, authentification JWT et déploiement Docker.

Frontend

Interface React réactive avec carte interactive

`React 18 + Vite + TypeScript`

Backend

API REST, authentification, logique métier

`Node.js + Express + TypeScript`

Base de données

Données cafés, utilisateurs, avis, favoris

`MongoDB + Mongoose`

Conteneurs

Orchestration dev + prod

`Docker Compose`

CI/CD

Validation, tests, sécurité, build

`GitLab Pipeline`

Supervision

Métriques, dashboards, disponibilité

`Prometheus · Grafana`

Données cartographiques

Import cafés + géocodage

`Overpass API + Nominatim`

Sécurité

Tokens signés, rôles, audit

`JWT + Gitleaks`

§ 02

Architecture système

Architecture client/API conventionnelle en trois couches. Le frontend gère l'état de présentation ; le backend gère l'authentification, la persistance, les règles de modération, l'ingestion de données externes, l'agrégation des avis et le scoring de recommandations.

```
Browser
  |  React 18 + Vite + React Router + TanStack Query
  |  fetch() with optional Bearer JWT
  v
Frontend
  | development : Vite :8081
  | production  : Nginx :80  (/api/* proxied to backend)
  v
Express API :3000
  |-- JWT middleware / optional-auth / admin guard
  |-- controllers (auth, cafes, favorites, ratings, admin)
```

```
  |-- Mongoose models (User, Cafe, Favorite, Rating)
  v
MongoDB :27017
  |-- 2dsphere index on Cafe.location
  |-- compound unique indexes (Favorite, Rating)

External Services:
  Overpass API  : radius café discovery + OSM import
  Nominatim     : address geocoding for submitted cafés
  Google Maps   : browser map display + directions

Observability:
  Backend /metrics --> Prometheus :9090 --> Grafana :3001
  cAdvisor + node-exporter --> Prometheus
```

Service	Adresse	Rôle
Application	`localhost:8081`	Interface utilisateur (dev)
Application	`localhost:80`	Interface utilisateur (prod)
API health	`localhost:3000/api/health`	Liveness check
Métriques	`localhost:3000/metrics`	Prometheus scrape
Prometheus	`localhost:9090`	Collecte métriques
Grafana	`localhost:3001`	Dashboards
Uptime Kuma	`localhost:3002`	Monitoring disponibilité

§ 03

Schémas DevOps

Les deux diagrammes suivants représentent l'architecture DevOps complète de Bean Beacon, du développeur à la production, incluant le pipeline GitLab CI/CD, l'environnement de conteneurs et la couche d'observabilité.

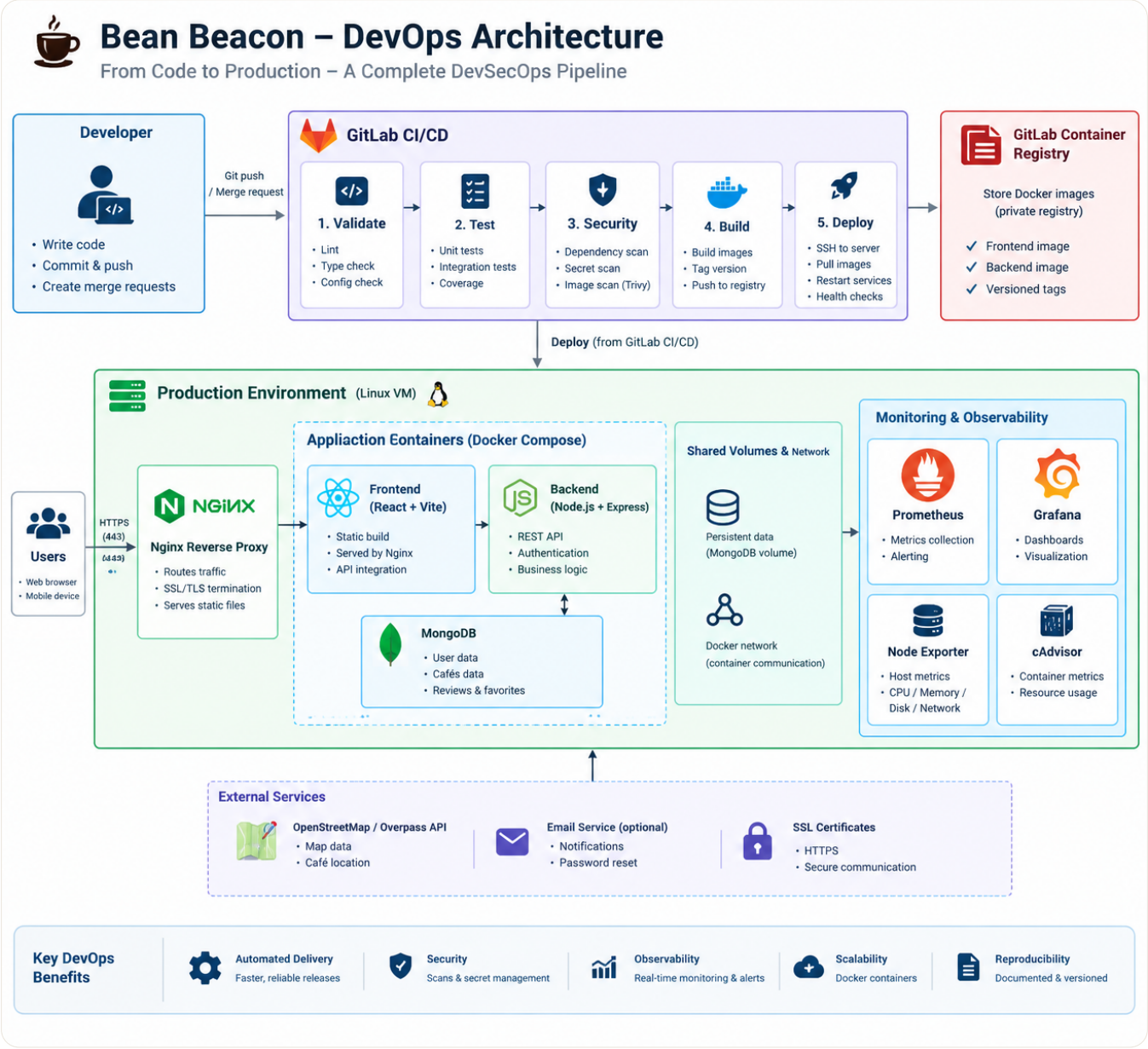


Fig. 1 — Architecture DevOps complète : du code à la production (vue claire)



Fig. 2 — Architecture DevOps complète : du code à la production (vue sombre)

PARTIE II

Mise en route

Prérequis, démarrage Docker Compose, tests et validations

§ 04

Prérequis

Docker Desktop avec Docker Compose — orchestration de tous les services

Git — versioning et intégration GitLab CI/CD

Node.js 16+ + npm — validations hors Docker (build, lint, tests)

Python + pyinfra — déploiement local automatisé via deploy.py

PowerShell — vérification

```
docker --version  # Docker version 24+
docker compose version  # Compose v2+
git --version
node --version  # v16+
npm --version
```

§ 05

Démarrage avec Docker Compose

PowerShell — démarrage développement

```
cd "C:\Users\msi\Desktop\coffee-quest\bean-beacon"

# Créer le réseau externe si nécessaire
docker network inspect bean-beacon_app-network *> $null
if ($LASTEXITCODE -ne 0) { docker network create bean-beacon_app-network }

# Valider et démarrer
docker compose config --quiet
docker compose up -d --build
docker compose ps
```

PowerShell — commandes utiles

```powershell
# Vérifier l'API
Invoke-RestMethod "http://localhost:3000/api/health"

# Logs en continu
docker compose logs --tail 100 -f backend
docker compose logs --tail 100 -f frontend

# Redémarrer un service
docker compose restart backend frontend

# Arrêter sans supprimer les volumes
docker compose stop
```

Préserver les données

Toujours utiliser `docker compose stop`. La commande `docker compose down -v` supprime définitivement les volumes MongoDB et Grafana.

§ 06

Tests et validations

PowerShell — backend

```powershell
cd bean-beacon-backend
npm ci
npm run build
npm test -- run  # Vitest
```

PowerShell — frontend

```powershell
cd ..ean-beacon-frontend
npm ci
npm run lint
npm run typecheck
npm run build
cd ..
```

PowerShell — scan sécurité

```powershell
docker run --rm -v "${PWD}:/path" zricethezav/gitleaks:latest detect --source="/path" --verbose --redact
```

PARTIE III

CI/CD et Déploiement

Workflow Git, pipeline GitLab et déploiement local de production

§ 07

Workflow Git et GitLab

La branche `main` doit rester déployable. Toute modification passe par une branche courte et une Merge Request avec revue humaine avant fusion dans `main`.

Git — cycle de contribution

```
git checkout -b feature/nom-de-la-fonctionnalite
git add .
git commit -m "feat: description de la modification"
git push origin feature/nom-de-la-fonctionnalite
# → Merge Request GitLab → revue → fusion dans main → pipeline
```

Revue humaine obligatoire

Le script `demo.ps1` ne crée pas de commit ni de Merge Request. Ces actions restent dans le workflow GitLab pour garantir une validation manuelle.

§ 08

Pipeline GitLab CI/CD

Le fichier `.gitlab-ci.yml` orchestre quatre étapes séquentielles. Le Runner doit porter le tag `bean-beacon`. Gitleaks est bloquant ; les audits npm sont en mode rapport. Harbor est exclu du périmètre.

ÉTAPE 01
validate
- Compilation TS backend
- Lint frontend
- TypeScript check
- Build frontend

→

ÉTAPE 02
test
- Tests unitaires
- Vitest backend

→

ÉTAPE 03
security
- npm audit frontend
- npm audit backend
- Gitleaks scan

→

ÉTAPE 04
build
- Image backend
- Image frontend
- Tag commit SHA

§ 09

Déploiement local de production

PowerShell — production

```powershell
# JWT obligatoire
$env:JWT_SECRET = "remplacer-par-un-secret-long-et-aleatoire"

# Docker Compose production
docker compose -f docker-compose.prod.yml up -d --build

# Ou via pyinfra
pyinfra @local deploy.py

# Application sur http://localhost
```

JWT_SECRET obligatoire en production

La variable doit être définie avant tout démarrage. Ne jamais utiliser la valeur de développement par défaut. Générer un secret unique avec `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`

Checklist post-déploiement

Vérification	Commande / URL
Frontend accessible	`http://localhost`
API health	`http://localhost:3000/api/health`
Inscription / connexion	POST `/api/auth/register`
Chargement des cafés	GET `/api/cafes/all`
HTTPS activé	Certificat Let's Encrypt
CORS configuré	`FRONTEND_URL` production
MongoDB en sauvegarde	Atlas backup ou cron

PARTIE IV

API Reference

Documentation complète des endpoints REST — Base URL : http://localhost:3000/api

§ 10

Authentification

Méthode	Endpoint	Auth	Description
POST	`/auth/register`	Public	Créer un compte, retourne JWT
POST	`/auth/login`	Public	Connexion, retourne JWT
GET	`/auth/me`	JWT	Profil utilisateur courant

Exemple — Login

```
# POST /api/auth/login
{
  "email": "user@example.com",
  "password": "securePassword123"
}

# Réponse 200
{
  "success": true,
  "data": {
    "token": "eyJhbGc...",
    "user": { "id": "507f...", "email": "user@example.com", "name": "John Doe" }
  }
}
```

§ 11

Cafés

Méthode	Endpoint	Auth	Description
GET	`/cafes/all`	Optionnel	Liste tous les cafés approuvés
GET	`/cafes?lat=&lng=&radius=`	Optionnel	Cafés à proximité (géospatial + OSM import)
GET	`/cafes/search?query=`	Optionnel	Recherche par nom / adresse
GET	`/cafes/:id`	Optionnel	Détail d'un café
POST	`/cafes`	JWT	Créer un café (géocodage auto si sans coord.)
POST	`/cafes/recommend`	Public	Recommandation par préférences (1–200 cafés)
PATCH	`/cafes/:id`	JWT	Modifier (propriétaire ou admin)
PATCH	`/cafes/:id/status`	Admin	Approuver / rejeter une soumission
DEL	`/cafes/:id`	JWT	Supprimer (propriétaire ou admin)

Découverte géospatiale

`GET /cafes` interroge d'abord MongoDB avec l'index 2dsphere. Si moins de 10 cafés, Overpass API est requêté, les nouveaux OSM cafés sont importés, puis MongoDB est relu. Le cache Overpass dure 1 heure.

§ 12

Favoris

Méthode	Endpoint	Auth	Description
GET	`/favorites`	JWT	Mes cafés favoris
POST	`/favorites/:cafeId`	JWT	Ajouter un favori
DEL	`/favorites/:cafeId`	JWT	Retirer un favori
GET	`/favorites/:cafeId/check`	Optionnel	Vérifier si favori

Backend complet, frontend non connecté

Les routes et modèles Favorite existent et fonctionnent en backend. Aucune méthode API n'est encore appelée depuis le frontend — aucun bouton favori n'est câblé.

§ 13

Évaluations & Avis

Méthode	Endpoint	Auth	Description
GET	`/cafes/:id/ratings`	Optionnel	Avis d'un café
POST	`/cafes/:id/ratings`	JWT	Créer / mettre à jour un avis (1–5 ★)
PUT	`/cafes/:id/ratings/:rid`	JWT	Modifier son avis
DEL	`/cafes/:id/ratings/:rid`	JWT	Supprimer (propriétaire ou admin)
GET	`/admin/reviews`	Admin	Tous les avis (max 500) pour modération

§ 14

Tableau de bord API complet

M.	Route	Auth	Description
GET	/health	Pub	Liveness check
GET	/metrics	Pub	Prometheus metrics
POST	/auth/register	Pub	Créer compte
POST	/auth/login	Pub	Connexion
GET	/auth/me	JWT	Profil courant
GET	/cafes/all	Opt	Tous les cafés approuvés
GET	/cafes	Opt	Géospatial + OSM import
GET	/cafes/search	Opt	Recherche texte
GET	/cafes/:id	Opt	Détail café
POST	/cafes	JWT	Créer café
POST	/cafes/recommend	Pub	Recommandation
PAT	/cafes/:id	JWT	Modifier café
PAT	/cafes/:id/status	ADM	Approuver / rejeter
POST	/cafes/:id/photo	JWT	Upload photo
DEL	/cafes/:id	JWT	Supprimer café
GET	/cafes/:id/ratings	Opt	Avis café
POST	/cafes/:id/ratings	JWT	Créer avis
PUT	/cafes/:id/ratings/:rid	JWT	Modifier avis
DEL	/cafes/:id/ratings/:rid	JWT	Supprimer avis
GET	/favorites	JWT	Mes favoris
POST	/favorites/:cafeId	JWT	Ajouter favori
DEL	/favorites/:cafeId	JWT	Retirer favori
GET	/favorites/:cafeId/check	Opt	Vérifier favori
GET	/admin/reviews	ADM	Tous les avis
GET	/admin/users	ADM	Liste utilisateurs
POST	/admin/users	ADM	Créer utilisateur
PAT	/admin/users/:id	ADM	Modifier utilisateur / rôle
PAT	/admin/cafes/:id	ADM	Admin modifier café

PARTIE V

Supervision, Livrables & Référence

Monitoring, améliorations, points d'attention et problèmes fréquents

§ 15

Supervision

Prometheus

Scrape `/metrics` toutes les 15 s. Collecte aussi cAdvisor et node-exporter.

`localhost:9090`

Grafana

Dashboards. Source Prometheus pré-configurée. Volume `grafana_data`.

`localhost:3001`

Uptime Kuma

Moniteur HTTP sur `/api/health` — 60 s, 2 retries. URL : `http://backend:3000/api/health`

`localhost:3002`

Métriques Prometheus exposées par le backend : `bean_beacon_http_request_duration_seconds` (histogram), `bean_beacon_http_requests_total` (counter), labellisées par méthode, route et code HTTP.

Réinitialisation Grafana

```
docker compose exec grafana grafana cli admin reset-admin-password "NouveauMotDePasseFort!"
```

§ 16

Livrables finaux

Frontend bean-beacon-frontend/	**Backend API** bean-beacon-backend/	**Base de données** Service mongo (Compose)
Compose dev docker-compose.yml	**Compose prod** docker-compose.prod.yml	**Pipeline GitLab** .gitlab-ci.yml
Health check GET /api/health	**Tests backend** bean-beacon-backend/tests/	**Déploiement pyinfra** deploy.py
Démo Windows demo.ps1	**Supervision** Prometheus · Grafana · Uptime Kuma	**Documentation** docs/FINAL_DOCUMENTATION.md

§ 17

Améliorations fonctionnelles

Analyse issue du code source existant — deux catégories : les fonctionnalités à moitié construites (victoires rapides) et les fonctionnalités entièrement manquantes.

Victoires rapides — backend déjà prêt

QUICK WIN

Favoris — backend complet, frontend non câblé

Le modèle Favorite, `favoriteController.ts` (get/add/remove) et les routes GET/POST/DELETE `/api/favorites` existent et fonctionnent. Aucune méthode API n'est appelée depuis le frontend — aucun bouton "sauvegarder" ni page "Mes favoris".

> **Pour finir :** ajouter les méthodes `favoritesApi` dans `api.ts`, une icône cœur/sauvegarde sur les cartes café, et une vue "Mes Favoris" (nouvelle page ou onglet filtre).

QUICK WIN

Upload de photos — champs schema existants, rien d'autre

Le modèle Cafe possède les champs `photoPath` et `galleryPaths`, mais il n'y a pas d'endpoint d'upload, pas de middleware multer, et aucun input fichier dans `Manage.tsx`.

> **Pour finir :** endpoint upload (stockage disque local), input fichier dans le formulaire de soumission, rendu de la photo dans `CafeProfile.tsx` et les cartes café.

Manquant entièrement

MANQUANT

Mot de passe oublié / flux de réinitialisation

`authRoutes.ts` n'a que `register`, `login` et `me` — aucun chemin de réinitialisation. Un utilisateur qui oublie son mot de passe ne peut pas récupérer son compte.

> **Périmètre :** `POST /api/auth/forgot-password` (token à usage unique, lien email) + `POST /api/auth/reset-password` (vérification token, nouveau hash). Service email : Resend ou console.log en dev.

MANQUANT

Notification propriétaire lors de l'approbation / rejet

Après soumission d'un café, l'utilisateur doit vérifier lui-même dans `Manage.tsx` si son café a été approuvé. Aucun email, aucune notification in-app.

> **Version minimale :** cloche/badge in-app lisant les changements de statut des cafés de l'utilisateur (pas d'infra supplémentaire). Version email : partage l'infra du reset de mot de passe ci-dessus.

MANQUANT

Tri par distance + pagination sur la liste principale

`GET /api/cafes/all` trie par `createdAt` uniquement et plafonne à 200 résultats sans paramètres `page` / `limit`. Le tri géospatial `$near` de `getCafesByLocation` n'est pas utilisé sur cet endpoint.

> **Périmètre :** ajouter page/limit à `listAllCafes`, et quand `lat`/`lng` sont passés, trier par distance via l'index 2dsphere déjà présent sur le modèle Cafe.

§ 18

Points d'attention & risques

#	Risque	Criticité
1	Frontend ne consomme pas l'API favoris	Moyenne
2	Bouton localisation obtient coords mais n'appelle pas `/api/cafes`	Faible
3	JWT stocké en localStorage (vulnérable XSS)	Haute
4	Pas de rate limiting ni pagination sur la plupart des vues publiques	Moyenne
5	Mise à jour agrégat notation non transactionnelle (race condition)	Faible
6	`JWT_SECRET` a un fallback dev — déploiements non-Compose à risque	Haute
7	Prometheus/Grafana absents du Compose production	Faible
8	Cache Overpass/Nominatim local — perdu au redémarrage, non partagé	Faible
9	Aucune validation schéma centralisée (Zod/Joi) côté backend	Moyenne
10	Pas de flux mot de passe oublié	Haute (UX)

§ 19

Problèmes fréquents

Aucun conteneur avec `docker compose ps`

La stack n'est pas démarrée.

Fix

```
docker compose up -d --build
```

API retourne `Route not found` sur `http://localhost:3000/`

Comportement normal — la route racine n'est pas définie. Utiliser `http://localhost:3000/api/health`.

Grafana refuse le mot de passe

Réinitialiser sans supprimer le volume :

```
Fix
docker compose exec grafana grafana cli admin reset-admin-password "NouveauMotDePasseFort!"
```

Perte des données MongoDB ou Grafana

Toujours utiliser `docker compose stop`. La commande `docker compose down -v` supprime définitivement les volumes persistants.

Jobs GitLab restent en attente

Vérifier que le GitLab Runner est enregistré avec le tag exact `bean-beacon`.

Cafés ne se chargent pas

Vérifier l'accès à Overpass API, la permission de géolocalisation, et que le backend peut joindre `overpass-api.de`. Consulter `docker compose logs -f backend`.